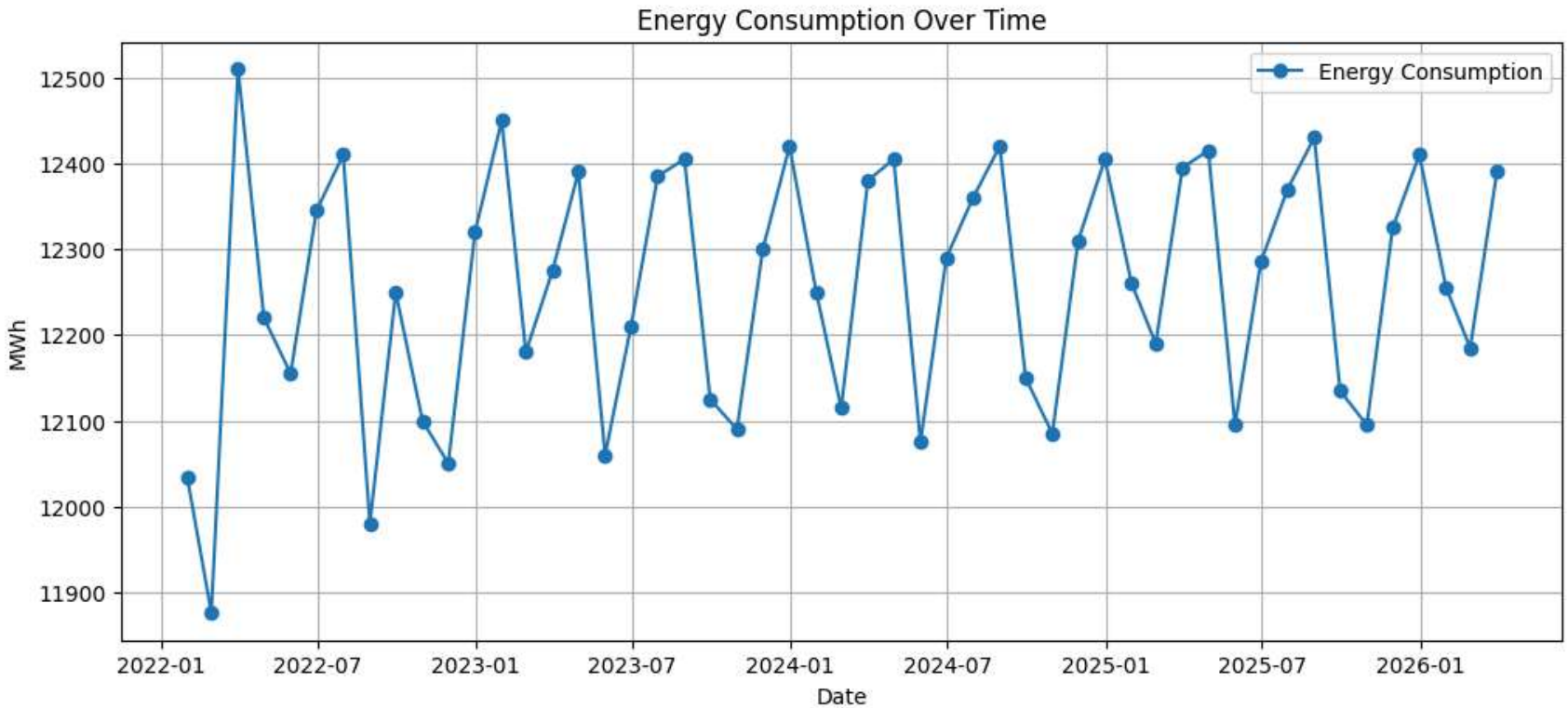
In [1]:

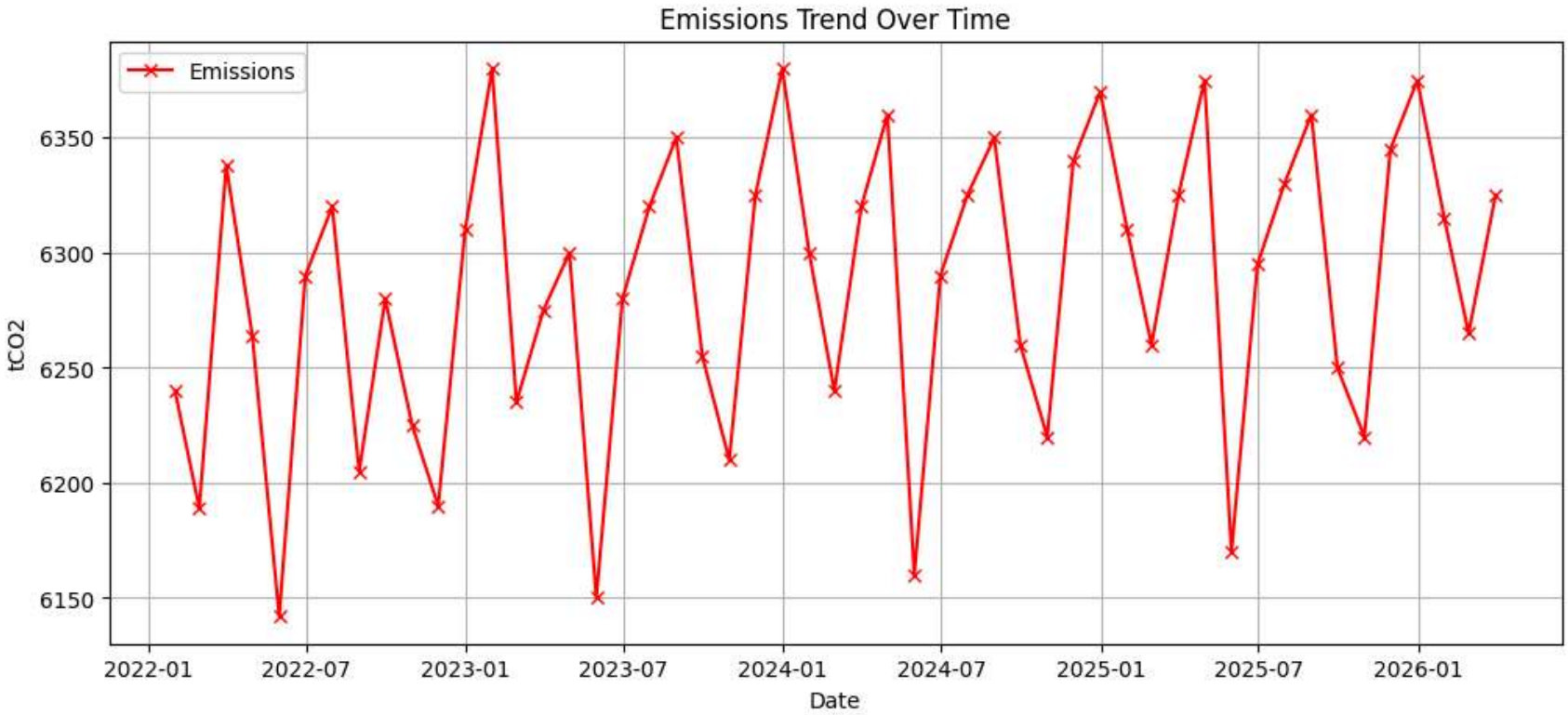
```python
# Project 1: Sustainability Forecasting & BI Dashboard (Refreshed)
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# ----------------------------
# Step 1: Load dataset from Excel
# ----------------------------
# Make sure the file is in the same folder
df = pd.read_excel("energy_consumption_data.xlsx", parse_dates=['Date'])
print("Dataset head:\n", df.head())

# ----------------------------
# Step 2: Basic cleaning
# ----------------------------
# Forward-fill missing values safely
df = df.fillna(df.ffill())

# Ensure fuel mix sums to 100%
df['Solar_Mix_%'] = 100 - df['Coal_Mix_%'] - df['Gas_Mix_%']

# ----------------------------
# Step 3: Visualization
# ----------------------------
plt.figure(figsize=(12,5))
plt.plot(df['Date'], df['Energy_Consumption_MWh'], marker='o', label='Energy Consumption')
plt.title("Energy Consumption Over Time")
plt.xlabel("Date")
plt.ylabel("MWh")
plt.grid(True)
plt.legend()
plt.show()

plt.figure(figsize=(12,5))
plt.plot(df['Date'], df['Emissions_tCO2'], marker='x', color='r', label='Emissions')
plt.title("Emissions Trend Over Time")
plt.xlabel("Date")
plt.ylabel("tCO2")
plt.grid(True)
```

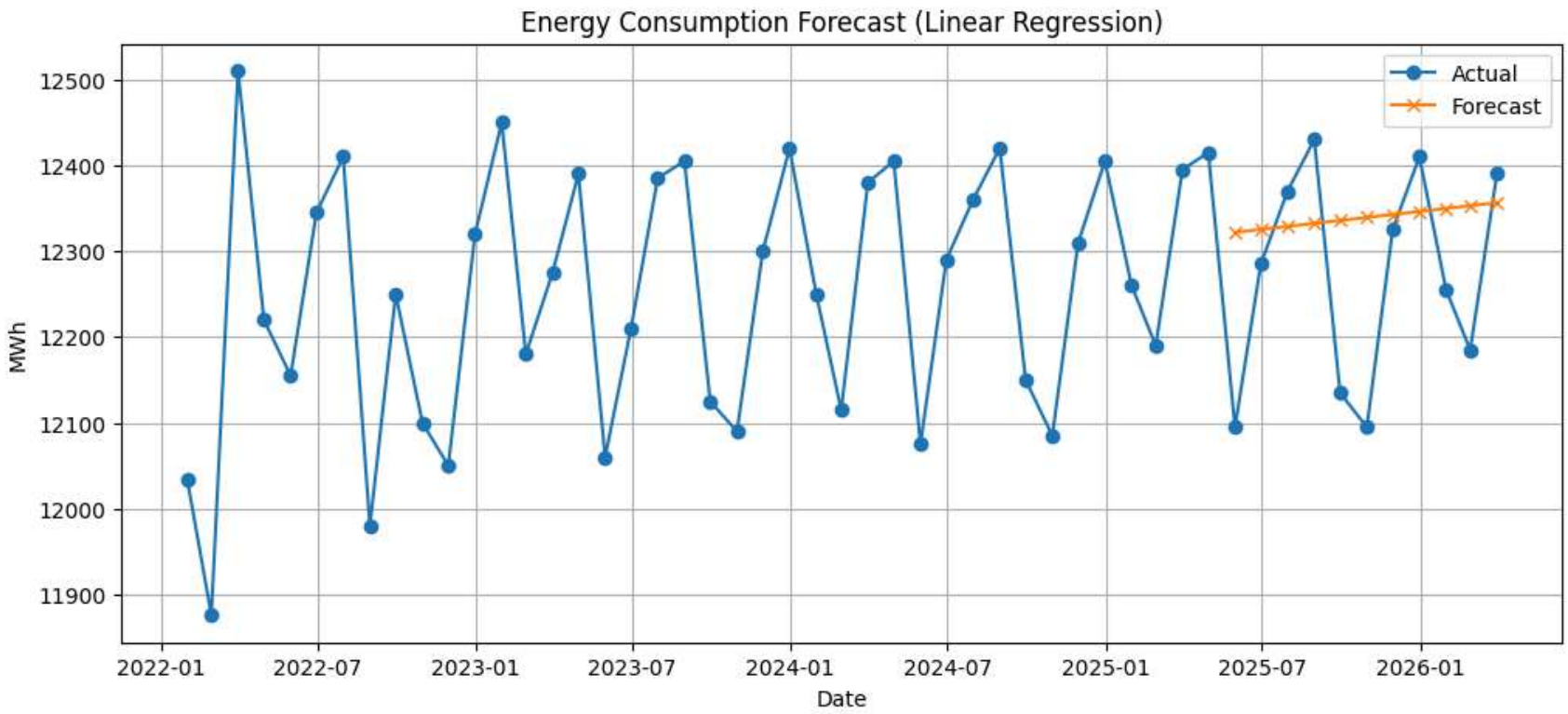
```python
plt.legend()
plt.show()

# ----------------------------
# Step 4: Simple Forecasting (Linear Regression)
# ----------------------------
# Convert date to ordinal for regression
df['Date_Ordinal'] = df['Date'].map(pd.Timestamp.toordinal)

X = df[['Date_Ordinal']]  # feature
y = df['Energy_Consumption_MWh']  # target

# Train-test split (keep order for time series)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)

# Fit linear regression
model = LinearRegression()
model.fit(X_train, y_train)

# Predict on test set
y_pred = model.predict(X_test)

# ----------------------------
# Step 5: Plot forecast vs actual
# ----------------------------
plt.figure(figsize=(12,5))
plt.plot(df['Date'], df['Energy_Consumption_MWh'], label='Actual', marker='o')
plt.plot(df['Date'].iloc[-len(y_test):], y_pred, label='Forecast', marker='x')
plt.title("Energy Consumption Forecast (Linear Regression)")
plt.xlabel("Date")
plt.ylabel("MWh")
plt.grid(True)
plt.legend()
plt.show()

# ----------------------------
# Step 6: Save forecast to Excel
# ----------------------------
forecast_df = pd.DataFrame({
    'Date': df['Date'].iloc[-len(y_test):],
    'Forecast_Energy_MWh': y_pred
})
```

```python
forecast_df.to_excel("energy_forecast.xlsx", index=False)
print("\nForecast Excel file saved as 'energy_forecast.xlsx'.")
```

```
Dataset head:
         Date  Energy_Consumption_MWh  Coal_Mix_%  Gas_Mix_%  Solar_Mix_%  \
0 2022-01-31                   12034          42         28           30
1 2022-02-28                   11876          40         33           27
2 2022-03-31                   12510          39         32           29
3 2022-04-30                   12220          44         26           30
4 2022-05-31                   12155          38         31           31

   Industrial_Demand_MWh  Emissions_tCO2
0                   7823            6240
1                   8102            6189
2                   7500            6338
3                   7980            6264
4                   7701            6142
```




Emissions Trend Over Time
Emissions
tCO2
6350
6300
6250
6200
6150
2022-01
2022-07
2023-01
2023-07
2024-01
2024-07
2025-01
2025-07
2026-01
Date




```
Forecast Excel file saved as 'energy_forecast.xlsx'.
```

In [3]:

```python
from matplotlib.backends.backend_pdf import PdfPages
import matplotlib.pyplot as plt

with PdfPages('Energy_Report.pdf') as pdf:
    plt.figure()
    plt.plot([1,2,3],[4,5,6])
    plt.title("Example Plot")
    pdf.savefig()
    plt.close()
```